

Problem Statement #31 — Multi-Vendor Artisan E-Commerce Marketplace

Lab 1: Requirements Engineering & UML Use-Case Modelling

1. Stakeholders / Actors

- Shopper — browses the marketplace, manages a cart, and places orders across one or more vendors.
- Artisan Vendor — manages a storefront, product catalog, order fulfillment, and views payouts.

2. Requirements Table

ID	Type	Description	Priorty	Acceptance Criteria	Rationale
FR-001	Functional	The system shall split customer cart payments at checkout, allocating respective item earnings to multiple independent vendor accounts after deducting a 5% platform commission fee.	High	Pass: Sum of individual vendor payouts plus platform commission equals total cart value across all test transactions. Fail: Any payout calculation discrepancy or missing vendor allocation.	Core marketplace value proposition depends on accurate, trustworthy automated revenue distribution among independent vendors without manual reconciliation.
FR-002	Functional	The system shall allow an Artisan Vendor to create, edit, and remove product listings in their storefront catalog, including title, description, price, inventory count, and up to 10 images per listing.	High	Pass: Vendor-submitted catalog changes are reflected in the live storefront within 60 seconds and validation rejects listings missing required fields. Fail: Changes are lost, delayed beyond 60 seconds, or invalid listings are published.	Vendors need direct, self-service control over their catalog to keep inventory and pricing accurate without platform administrator intervention.
FR-003	Functional	The system shall allow a Shopper to add products from multiple different vendors into a single shopping cart and complete checkout as one combined order.	High	Pass: A checkout containing items from 2+ vendors generates one order confirmation and correctly itemizes sub-orders per vendor. Fail: Multi-vendor checkout fails, splits into separate uncombined transactions, or misattributes items.	A unified cart across independent vendors is the core UX differentiator of a multi-vendor marketplace versus single-seller stores.
FR-004	Functional	The system shall notify the Shopper of order status changes (Confirmed, Shipped, Out for Delivery, Delivered) via in-app notification and email, and allow the Shopper to view real-time order status.	Medium	Pass: Status-change events trigger a notification within 5 minutes and the order status page reflects the latest state. Fail: Notification is not sent or the status page shows a stale/incorrect state.	Order visibility builds shopper trust, which is especially important in an artisan marketplace with variable, vendor-specific fulfillment times.
FR-005	Functional	The system shall generate a downloadable payout report for each Artisan Vendor summarizing	Medium	Pass: Report totals reconcile exactly with the underlying transaction ledger for the	Vendors require transparent financial reporting to reconcile

ID	Type	Description	Priority	Acceptance Criteria	Rationale
		gross sales, platform commission deducted, and net payout for a selected date range.		selected period. Fail: Report totals do not match transaction records, or report generation fails.	earnings, file taxes, and trust the platform's commission deductions.
NFR-001	Performance & Security	The product catalog must support high-resolution image rendering with CDN caching delivering load times < 500 ms.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load. Fail: Load time exceeds 500 ms or security benchmarks are not met.	Fast, secure catalog browsing is essential to shopper conversion and protects vendor and shopper data from exposure.
NFR-002	Reliability & Availability	The system shall maintain at least 99.9% monthly uptime for checkout and payment-processing services; any failed vendor payout transaction shall automatically retry up to 3 times with rollback and admin alerting on final failure.	High	Pass: Uptime monitoring confirms $\geq 99.9\%$ availability over a monthly window, and simulated payout failures correctly retry/rollback without duplicate or lost funds. Fail: Uptime falls below threshold, or a failed payout causes duplicate charges, lost funds, or a silent failure.	Real money and vendor livelihoods depend on payout accuracy, so the payment pipeline must be highly available and fail safely rather than silently.

3. Use-Case Flow Specification

Use Case: Checkout Cart with Split Payment

Primary Actor: Shopper

Secondary Actors: Artisan Vendor (payout recipient), Payment Gateway (external system)

Preconditions

- The Shopper is authenticated and logged in.
- The Shopper's cart contains at least one item from at least one Artisan Vendor.
- The Shopper has a valid, verified payment method on file.
- All items in the cart are currently in stock.

Postconditions

- Success: The order is created and split into per-vendor sub-orders; payment is captured; the 5% platform commission is deducted; each vendor's ledger is credited with its net payout; inventory is decremented; and confirmations are sent to the Shopper and each involved Vendor.
- Failure: No charge is made, the cart remains unchanged, and the Shopper is informed of the failure reason.

Main Success Scenario

1. The Shopper navigates to the Cart and selects “Proceed to Checkout.”
2. The system displays an order summary grouped by vendor, including an itemized subtotal per vendor and the total cart value.
3. The Shopper confirms a shipping address and selects a payment method.
4. The Shopper selects “Place Order.”
5. The system validates the cart (stock availability and price accuracy).
6. The system «includes» the Process Split Payment use case, charging the Shopper's payment method for the total cart value via the Payment Gateway.
7. The system calculates each vendor's net payout (item subtotal minus the 5% platform commission) and credits it to that vendor's pending payout ledger.
8. The system decrements inventory for each purchased item and creates per-vendor sub-orders under one parent order.
9. The system displays an order confirmation to the Shopper and sends confirmation notifications to the Shopper and each involved Artisan Vendor.

Alternate Flow A1 — Payment Declined (branches at step 6)

- 6a. The Payment Gateway declines the charge.
- 6b. The system displays a payment-failure message to the Shopper and does not create the order or credit any vendor ledger.
- 6c. The Shopper is prompted to select a different payment method or retry.
- 6d. If the Shopper provides a new valid payment method, the flow resumes at step 4; otherwise the use case ends in failure and the cart is preserved.

Extend: Apply Discount Code

Optionally extends this use case between steps 3 and 4 if the Shopper enters a valid promo code, adjusting the displayed total before payment is processed.

